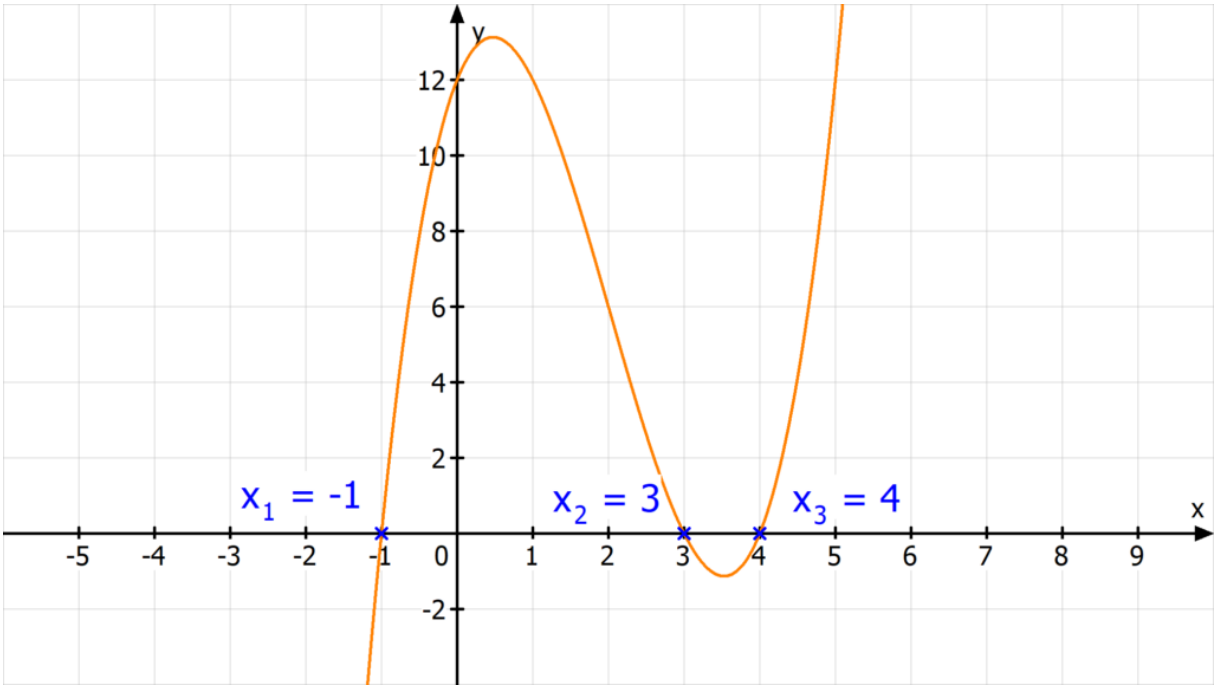
Bisektion-Projekt

Implementierung und Analyse der
Bisektions- und Regula-Falsi-Methode

Name: Marin Pravdić
Klasse: 3BHWII
Schuljahr: 2025/26
Gegenstand: Softwareentwicklung & Projektmanagement (SWP)



Allgemeines Ziel

Ziel dieses Projekts ist die numerische Bestimmung von Nullstellen stetiger Funktionen mit einer Variablen x. Dazu wurden verschiedene iterative Verfahren implementiert und getestet. Zusätzlich wurden die Ergebnisse grafisch dargestellt und auf ein reales Problem angewendet.

Aufgabe 5 – Bisektion

Das Bisektionsverfahren ist ein numerisches Verfahren zur Bestimmung von Nullstellen. Dabei wird ein Intervall [a; b] gewählt, in dem ein Vorzeichenwechsel vorliegt. Das Intervall wird iterativ halbiert.

Formel:

$$c = \frac{a + b}{2}$$

Vorgehen

- Berechnung des Mittelpunkts c
- Überprüfung, in welchem Teilintervall die Nullstelle liegt
- Wiederholung bis zur gewünschten Genauigkeit ε

Die Nullstele wird iterativ bestimmt und nähert sich mit steigender Iterationszahl dem exakten Wert an.

```
# Hauptschleife der Bisektionsmethode, die so lange läuft, bis der Funktionswert an c kleiner als epsilon ist
while abs(fc) >= epsilon:
    # Überprüfen, ob einer der Intervallgrenzen oder die Mitte die Nullstelle ist
    if fa == 0:
        return a, fa, error_values, x_values, iterations
    elif fb == 0:
        return b, fb, error_values, x_values, iterations
    elif fc == 0:
        return c, fc, error_values, x_values, iterations

    # Bestimmen, in welchem Intervall die Nullstelle liegt, und die Intervallgrenzen entsprechend anpassen
    if fa * fc < 0:
        b = c
        fb = fc
    else:
        a = c
        fa = fc

    # Berechnung der neuen Mitte des Intervalls und des Funktionswerts an dieser Stelle
    c = (a + b) / 2
    fc = function(c, function_str)

    error_values.append(abs(fc))
    x_values.append(c)

    iterations += 1

return c, fc, error_values, x_values, iterations
```

Ausgabe

```
PS C:\Users\m4r1n\OneDrive - tgm - Die Schule der Technik\TGM\3. Jahrgang\SWP\SWP\Projekt\Programmierung\bisektion-MarinPravdic> python Aufgabe5.py

Test für n = 25
Numerisch: 5.000000000291038
Analytisch: 5.0
Ungenauigkeit: 2.9103830456733704e-10

Test für n = 81
Numerisch: 8.999999999869033
Analytisch: 9.0
Ungenauigkeit: 1.3096723705530167e-10

Test für n = 144
Numerisch: 12.000000000349246
Analytisch: 12.0
Ungenauigkeit: 3.4924596548080444e-10

Bitte geben Sie die Funktion f(x) ein (z.B. 'x**2 - 4'): x**2-16

Bitte geben Sie die erste Intervallsgrenze ein: 2
Bitte geben Sie die zweite Intervallsgrenze ein: 5

Bitte geben Sie die gewünschte Genauigkeit ein (10^...): -16

Die Nullstelle liegt näherungsweise bei: 4.0, Funktionswert: 0.0
```

Aufgabe 6 – Regula falsi

Das Regula-Falsi-Verfahren ist eine Erweiterung der Bisektion. Statt der Intervallhalbierung wird eine lineare Approximation verwendet. Der einzige Unterschied zur Bisektion ist die Verwendung einer anderen Formel.

Formel:

$$c = b - f(b) \cdot \frac{b - a}{f(b) - f(a)}$$

Vorgehen

- Berechnung des Schnittpunkts c der Sekante mit der x-Achse
- Überprüfung, in welchem Teilintervall die Nullstelle liegt
- Anpassung des Intervalls
- Wiederholung bis zur gewünschten Genauigkeit ε

Im Vergleich zur Bisektion konvergiert dieses Verfahren oft schneller, da es die Funktionswerte berücksichtigt.

```python
# Hauptschleife der Regula-Falsi-Methode, die so lange läuft, bis der Funktionswert an c kleiner als epsilon ist
while abs(fc) >= epsilon:
    # Überprüfen, ob einer der Intervallgrenzen oder die Mitte die Nullstelle ist
    if fa == 0:
        return a, fa, error_values, x_values
    elif fb == 0:
        return b, fb, error_values, x_values
    elif fc == 0:
        return c, fc, error_values, x_values

    # Bestimmen, in welchem Intervall die Nullstelle liegt und die Intervallgrenzen entsprechend anpassen
    if fa * fc < 0:
        b = c
        fb = fc
    else:
        a = c
        fa = fc

    # Einziger Unterschied zur Bisektionsmethode: Berechnung der neuen Stelle c anhand der Regula-Falsi-Formel
    c = b - fb * ((b - a) / (fb - fa))
    fc = function(c, function_str)

    error_values.append(abs(fc))
    x_values.append(c)

return c, fc, error_values, x_values
```

Die Nullstelle wird schneller erreicht als bei der Bisektion, besonders bei gut gewählten Intervallen.

Ausgabe

```
PS C:\Users\m4r1n\OneDrive - tgm - Die Schule der Technik\TGM\3. Jahrgang\SWP\SWP\Projekt\Programmierung\bisektion-MarinPravdic> python Aufgabe6.py

Test für n = 16
Numerisch: 3.999999999168434
Analytisch: 4.0
Ungenauigkeit: 8.315659272284393e-10

Test für n = 15
Numerisch: 3.8729833451549265
Analytisch: 3.872983346207417
Ungenauigkeit: 1.0524905391662287e-09

Test für n = 25
Numerisch: 4.999999999082096
Analytisch: 5.0
Ungenauigkeit: 9.17903975050649e-10

Test für n = 81
Numerisch: 8.999999999507864
Analytisch: 9.0
Ungenauigkeit: 4.921361096421606e-10

Test für n = 144
Numerisch: 11.99999999962833
Analytisch: 12.0
Ungenauigkeit: 3.716706942213932e-10

Bitte geben Sie die Funktion f(x) ein (z.B. 'x**2 - 4'): x**2-16

Bitte geben Sie die erste Intervallsgrenze ein: 2
Bitte geben Sie die zweite Intervallsgrenze ein: 5

Bitte geben Sie die gewünschte Genauigkeit ein (10^...): -16

Die Nullstelle liegt näherungsweise bei: 4.0, Funktionswert: 0.0
```

Aufgabe 7

Ziel dieser Aufgabe ist die grafische Darstellung der Nullstellenfindung mittels der zuvor implementierten Verfahren (Bisektion und Regula-Falsi).

Dabei soll der Iterationsprozess visualisiert werden, um das Konvergenzverhalten besser nachvollziehen zu können.

Vorgehen

Zunächst werden die Eingaben des Benutzers abgefragt:

- Funktion f(x)
- Intervall [a; b]
- Genauigkeit ε
- gewünschtes Verfahren (Bisektion oder Regula Falsi)

Anschließend wird die Nullstelle mit dem gewählten Verfahren berechnet. Während der Iteration werden zusätzlich folgende Werte gespeichert:

- Genauigkeit: $|f(c)|$
- aktuelle Näherungen: c

Diese Werte werden für die grafische Darstellung verwendet.

Visualisierung

Zur Darstellung werden zwei Diagramme (Subplots) verwendet:

1. Genauigkeit

- Darstellung von $|f(c)|$ über die Iterationen
- zeigt, wie sich die Genauigkeit dem Wert ε annähert

2. Aktuelle Lösung

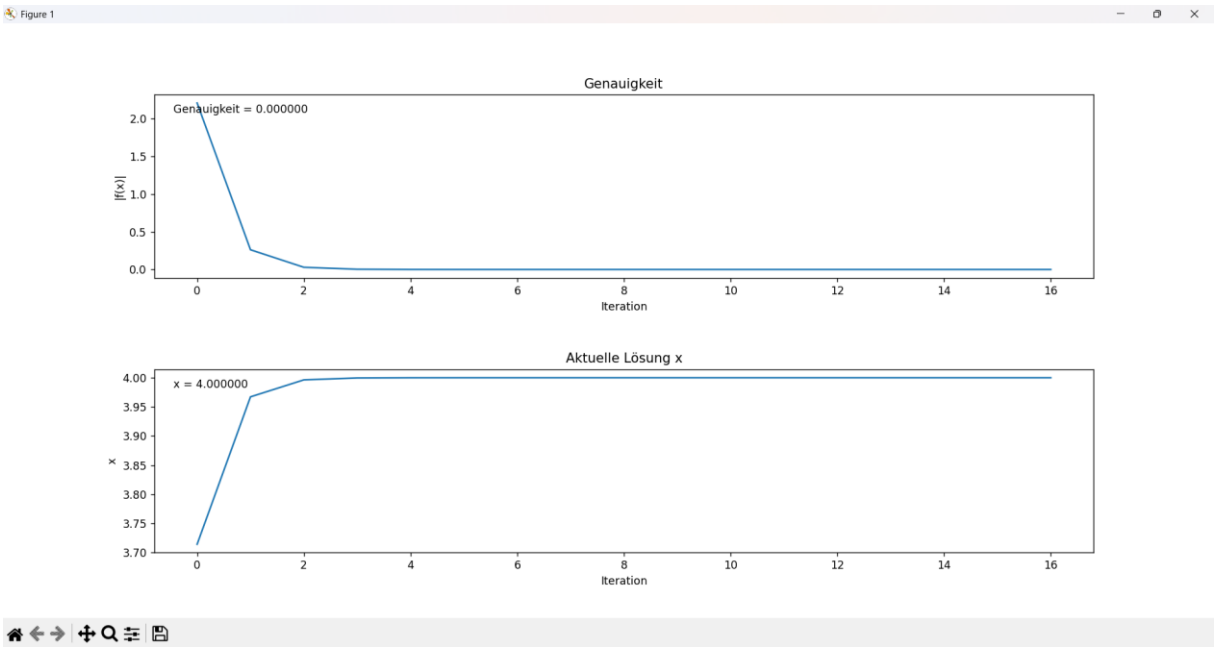
- Darstellung der Näherung c über die Iterationen
- zeigt, wie sich der berechnete Wert der tatsächlichen Nullstelle annähert

Die Diagramme werden während der Berechnung schrittweise aktualisiert, wodurch eine Animation entsteht.

```python
def visualisation(accuracy_values : list, x_values : list) -> None:

    Parameter:
        - accuracy_values (list): Liste der Fehlerwerte (|f(c)|) für jede Iteration
        - x_values (list): Liste der aktuellen Lösungen c für jede Iteration

    Rückgabewerte:
        - None
    """

    fig, (graph1, graph2) = plt.subplots(2, 1, figsize=(16, 8))     # Erstellen von zwei Subplots: ein
    fig.subplots_adjust(hspace=0.5)                                 # Abstand zwischen den Subplots an

    # Animieren der Plots, indem in jeder Iteration die Fehlerwerte und aktuellen Lösungen bis zu dies
    for i in range(len(accuracy_values)):
        # Plots löschen, um die neuen Werte zu aktualisieren
        graph1.clear()
        graph2.clear()

        # Fehlerverlauf und aktuelle Lösung bis zur aktuellen Iteration plotten
        graph1.plot(accuracy_values[:i+1])
        graph1.set_title("Genauigkeit")
        graph1.text(0.02, 0.9, f"Genauigkeit = {accuracy_values[i]:.6f}", transform=graph1.transAxes)
        graph1.set_xlabel("Iteration")
        graph1.set_ylabel("|f(x)|")

        # Aktuelle Lösung x bis zur aktuellen Iteration plotten
        graph2.plot(x_values[:i+1])
        graph2.set_title("Aktuelle Lösung x")
        graph2.text(0.02, 0.9, f"x = {x_values[i]:.6f}", transform=graph2.transAxes)            # Aktu
        graph2.set_xlabel("Iteration")
        graph2.set_ylabel("x")

        plt.pause(0.1)  # Kurze Pause, um die Animation sichtbar zu machen

    plt.show()
```

Ausgabe

```
PS C:\Users\m4r1n\OneDrive - tgm - Die Schule der Technik\TGM\3. Jahrgang\SWP\SWP\Projekt\Programmierung\bisektion-MarinPravdic> python Aufgabe7.py

Bitte geben Sie die Funktion f(x) ein (z.B. 'x**2 - 4'): x**2-16

Bitte geben Sie die erste Intervallsgrenze ein: 2
Bitte geben Sie die zweite Intervallsgrenze ein: 5

Bitte geben Sie die gewünschte Genauigkeit ein (10^...): -16

Möchten Sie die Bisektion-Methode oder die Regula-Falsi-Methode verwenden? (b/r): r

Die Nullstelle liegt näherungsweise bei: 4.0, Funktionswert: 0.0
```



Aufgabe 8

Es wird ein Polynom vierten Grades betrachtet:

$$P_4(x) = 2x + x^2 + 3x^3 - x^4$$

Ziel ist es, mit dem implementierten Bisektionsverfahren die Nullstelle zu bestimmen und den Einfluss der Genauigkeit ε auf die Anzahl der Iterationen zu untersuchen.

Vorgehen

Zur Bestimmung der Nullstelle wird ein geeignetes Intervall gewählt:

$$[a;\ b] = [2;\ 5]$$

Dieses Intervall enthält eine Nullstelle, da ein Vorzeichenwechsel der Funktion vorliegt.

Die Berechnung wird für zwei unterschiedliche Genauigkeiten durchgeführt:

- $\varepsilon = 10^{-2}$
- $\varepsilon = 10^{-8}$

Die Iteration wird jeweils so lange durchgeführt, bis die Abbruchbedingung erfüllt ist:

$$|\ f(c)\ | < \varepsilon$$

Dabei wird zusätzlich die Anzahl der benötigten Iterationen gezählt.

Ergebnisse

Genauigkeit ε	Iterationen
10^{-2}	11
10^{-8}	29

Zusätzlich wird die Abweichung von der tatsächlichen Nullstelle x = 3,4567 berechnet.

Die Ergebnisse zeigen, dass mit steigender Genauigkeit ε die Anzahl der Iterationen deutlich zunimmt.

Dies liegt daran, dass das Bisektionsverfahren das Intervall in jedem Schritt halbiert. Um eine höhere Genauigkeit zu erreichen, muss das Intervall somit häufiger geteilt werden.

Für eine grobe Genauigkeit ($\varepsilon = 10^{-2}$) wird die Nullstelle relativ schnell gefunden, während bei einer sehr hohen Genauigkeit ($\varepsilon = 10^{-8}$) deutlich mehr Iterationen notwendig sind.

Ausgabe

```
PS C:\Users\m4r1n\OneDrive - tgm - Die Schule der Technik\TGM\3. Jahrgang\SWP\SWP\Projekt\Programmierung\bisektion-MarinPravdic> python Aufgabe8.py

Die Nullstelle liegt näherungsweise bei: 3.456787109375, Funktionswert: -0.005303831051378438
Anzahl der Iterationen für Genauigkeit von 0.01: 11
Abweichung von der tatsächlichen Nullstelle: 8.710937499989413e-05

Die Nullstelle liegt näherungsweise bei: 3.4566783430054784, Funktionswert: 1.8837056359188864e-09
Anzahl der Iterationen für Genauigkeit von 1e-08: 29
Abweichung von der tatsächlichen Nullstelle: 2.165699452172376e-05
```

Aufgabe 9

Ziel dieser Aufgabe ist die Anwendung der zuvor implementierten numerischen Verfahren auf ein reales Problem.

Es soll die Länge einer elektrischen Leitung berechnet werden, die zwischen zwei Masten mit einem Abstand von 100 m gespannt ist und in der Mitte einen Durchhang von 10 m aufweist.

Berechnung

22:35 Donnerstag 30. Apr. 48 %

SWP

Start Einfügen Zeichnen Ansicht

Rechtschreibung Hintergrund wechseln Plastischer Reader 100 % Seitenbre

Aufgabe 9

Mittwoch, 29. April 2026 17:29

$w = 100m$

$\text{Durchhang} = 10m$

Formeln:

- Länge:

$$L = 2a \cdot \sinh\left(\frac{w}{2a}\right)$$

- Kettenliniengleichung:

$$y(x) = a \cdot \cosh\left(\frac{x - x_0}{a}\right) - a + y_0 \quad ; \quad x_0 = 0$$

- Strommasthöhe:

$$y(50m) = y_0 + 10m$$

Berechnung:

$$y(50m) = a \cdot \cosh\left(\frac{50m}{a}\right) - a + y_0$$

$$y(50m) = y_0 + 10m$$

$$a \cdot \cosh\left(\frac{50m}{a}\right) - a + y_0 = y_0 + 10m \quad | -y_0$$

$$a \cdot \cosh\left(\frac{50m}{a}\right) - a = 10m \quad | -10$$

$$a \cdot \cosh\left(\frac{50m}{a}\right) - a - 10m = 0$$

$[1; 1000]$; $\varepsilon = 10^{-16}$; Bisektion

$\hookrightarrow \underline{\underline{a \approx 126{,}63m}}$

$$L = 2a \cdot \sinh\left(\frac{50}{a}\right)$$

$$L = 2 \cdot 126{,}63 \cdot \sinh\left(\frac{50}{126{,}63}\right)$$

$$\underline{\underline{L \approx 102{,}62m}}$$

Vorgehen

Zunächst wird die Gleichung der Kettenlinie in eine Form gebracht, die als Nullstellenproblem gelöst werden kann:

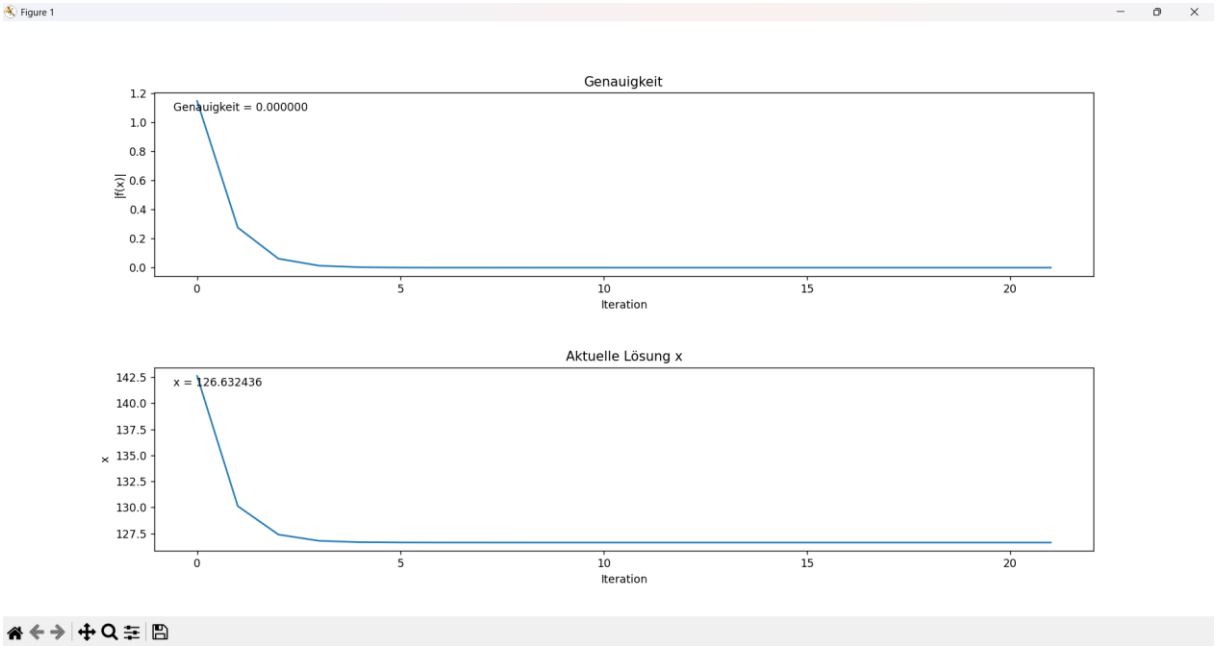
$$f(x) = x \cdot cosh\left(\frac{50}{x}\right) - x - 10$$

Anschließend wird ein geeignetes Intervall gewählt:

$$[a;\ b] = [100;\ 200]$$

Dieses Intervall enthält die gesuchte Lösung für den Krümmungsradius.

Mit dem Bisektionsverfahren oder dem Regula-Falsi-Verfahren wird die Nullstelle berechnet.

Ausgabe

```
PS C:\Users\m4r1n\OneDrive - tgm - Die Schule der Technik\TGM\3. Jahrgang\SWP\SWP\Projekt\Programmierung\bisektion-MarinPravdic> python Aufgabe9.py

Bitte geben Sie die gewünschte Genauigkeit ein (10^...): -16

Möchten Sie die Bisektion-Methode oder die Regula-Falsi-Methode verwenden? (b/r): r

Die Nullstelle liegt näherungsweise bei: 126.63243603998896, Funktionswert: 0.0

Der Krümmungsradius a beträgt: 126.63243603998896m
Die Länge der elektrischen Leitung beträgt: 102.61868681287588m
```